

- Método:** `uma_linha`
Descrição: Retorna uma linha do placar, a linha 9 recebe uma formatação especial.
Domínio de entrada: Posição válida no placar (inteiros positivos).
Domínio de saída: String representando a linha do placar.
Classes inválidas: $i > 10$, $i < 0$.
Classes válidas: Qualquer valor de i entre 0 e 9 que seja inteiro.
- Método:** `add`
Descrição: Adiciona a pontuação dos dados obtida dos dados rolados em uma posição no placar.
Domínio de entrada: Posição válida no placar (inteiros positivos, maiores ou iguais a 1 e menores que 10) e dados válidos (uma lista de inteiros positivos de tamanho igual a 5). **Domínio de saída:** `IndexError` para posições menores que 1 e maiores que 10; `ValueError` para posições já selecionadas; `IndexError` para dados com tamanho menor que 5.
Classes inválidas: $i > 9$ ou $i < 0$. Casos de teste: $\{ [1000, 6], [-5, 2], ([6, 3], [6, 2], [2, 10]) \}$. Importante: o caso $([6, 3], [6, 2])$ representa a seleção repetida de uma posição válida.
Classes válidas: Posição de $[0, 9]$, casos de teste: $\{ 2, 5, 9, 3 \}$. Lista de dados de tamanho 5, casos de teste: $\{ [2, [5,2,3,4,5]], [6, [5,2,3,4,6]], [1, [4,1,5,4,5]] \}$.
- Método:** `getScore`
Descrição: Retorna a pontuação total do placar ou a pontuação específica de uma posição no placar.
Domínio de entrada: Uma posição k inteira positiva $0 \leq k \leq 9$ ou um k nulo.
Domínio de saída: Um inteiro positivo ≥ 0 e ≤ 297 .
Classes inválidas: $k < 0$ ou $k > 9$. Casos de teste: $\{ -1, 100, -2000, -9, -10 \}$.
Classes válidas: $[0, 9]$. Casos de teste: $\{ 1, 5, 9, 0 \}$.
- Método:** `getTaken`
Descrição: Retorna se uma posição no placar foi selecionada ou não.
Domínio de entrada: Uma posição k inteira positiva $0 \leq k \leq 9$.
Domínio de saída: Um valor booleano `True` ou `False`.
Classes inválidas: $k < 0$ ou $k > 9$. Casos de teste: $\{ -1, 100, -2000, -9, -10 \}$.
Classes válidas: $[0, 9]$. Casos de teste: $\{ 1, 5, 9, 0 \}$.
- Método:** `getName`
Descrição: Retorna o nome de um tipo de pontuação (quadra, quina, full etc.).
Domínio de entrada: Uma posição k inteira positiva $0 \leq k \leq 9$.
Domínio de saída: Uma string com o nome do tipo de pontuação.
Classes inválidas: $k < 0$ ou $k > 9$. Casos de teste: $\{ -1, 100, -2000, -9, -10 \}$.
Classes válidas: $[0, 9]$. Casos de teste: $\{ 1, 5, 9, 0 \}$.
- Método:** `conta`
Descrição: Conta quantas vezes um número aparece em um vetor.
Domínio de entrada: Um número n e um vetor de números `vet`.
Domínio de saída: Um valor inteiro `cont` que representa a quantidade de vezes que n aparece em `vet`.

Classes inválidas: n não é um número ou vet não é um vetor de números. Casos de teste: { ('a', [1, 2, 3]), (2, [1, 2, 'a']), ('a', [1, 2, 3, 4, 5]), (null, [1, 2, 3, 4, 5, 6]) }.

Classes válidas: n é um número e vet é um vetor de números. Casos de teste: { (1, [1, 2, 5]), (6, [2, 1, 5]) }.

- **Método:** checkFull

Descrição: Verifica se uma combinação de 5 dados constitui um “Full House” (Trinca e Par).

Domínio de entrada: Um lista dados contendo 5 valores inteiros de dados.

Domínio de saída: Um valor booleano (True se for Full House, False caso contrário).

Classes inválidas: dados não é um lista de 5 inteiros, ou contém valores fora do intervalo [1, 6]. Casos de teste: { [1, 1,, 4, 1, 2, 2], [5, 5, 3, 6, 6, 6], [1, 2, 4, 5] }.

Classes válidas: dados é um lista de 5 inteiros. Casos de teste: { [1, 1, 1, 2, 2], [5, 5, 6, 6, 6], [1, 2, 3, 4, 5] }.

- **Método:** checkSeqMaior

Descrição: Verifica se uma combinação de 5 dados forma uma “Sequência Maior” (5 valores consecutivos, por exemplo: 2, 3, 4, 5, 6).

Domínio de entrada: Um lista dados contendo 5 valores inteiros de dados (1 a 6).

Domínio de saída: Um valor booleano (True se for Sequência Maior, False caso contrário).

Classes inválidas: dados não é um lista de 5 inteiros, ou contém valores fora do intervalo [1, 6].

Classes válidas: dados é um lista de 5 inteiros. Casos de teste: { [2, 3, 4, 5, 6], [1, 3, 2, 5, 4] (Sequência, deve ser True), [1, 2, 3, 4, 6] (Não é sequência, deve ser False) }.

- **Método:** checkQuadra

Descrição: Verifica se uma combinação de 5 dados contém uma “Quadra” (4 dados com o mesmo valor).

Domínio de entrada: Um vetor/lista “dados” contendo 5 valores inteiros de dados.

Domínio de saída: Um valor booleano (True se houver Quadra, False caso contrário).

Classes inválidas: dados não é um lista de 5 inteiros, ou contém valores fora do intervalo [1, 6].

Classes válidas: dados é um lista de 5 inteiros. Casos de teste: { [3, 3, 3, 3, 1], [6, 4, 4, 4, 4], [2, 2, 2, 1, 1] (Não é Quadra, deve ser False) }.

- **Método:** checkQuina

Descrição: Verifica se uma combinação de 5 dados constitui uma “Quina” (5 dados com o mesmo valor).

Domínio de entrada: Um vetor/lista “v” contendo 5 valores inteiros de dados.

Domínio de saída: Um valor booleano (True se for Quina, False caso contrário).

Classes inválidas: “v” não é um vetor/lista de 5 inteiros, ou contém valores fora do intervalo [1, 6].

Classes válidas: v é um vetor de 5 inteiros. Casos de teste: { [5, 5, 5, 5, 5], [1, 1, 1, 1, 1], [1, 2, 3, 4, 5] }

